From a PDF to a number

Learn PDF parsing by following one purchase

A beginner tutorial using the Portfolio app

Imagine you receive a PDF receipt that says **Net Amount Payable 340,728.50**. You can read it and enter the amount into a form yourself. Portfolio tries to fill in the form for you. To do that, it first gets the words from the PDF, then works out what those words mean.

```
On the page:      Net Amount Payable  340,728.50
As a line of text: "Net Amount Payable  340,728.50"
As a form value:   netAmountPayable = 340728.5
```

The first step is **text extraction**. It gets text out of a PDF or recognizes it from a picture of the page. The second step is **field parsing**. It finds a label, takes the value next to it, and converts the value into a form the app can use. A *field* is one named piece of data, such as the amount you owe or the date you bought shares.

What is a supported PDF? A file can open and look fine even if Portfolio has never learned its layout. Portfolio has rules for four kinds of investment documents:

- A BNR investor result confirming an individual bond purchase.
- A BNR public market results report describing a bond issue.
- A BK Capital equity purchase contract note for an RSE share purchase in Rwandan francs (RWF).
- A BK Capital client statement showing activity in RWF.

BNR is Rwanda's central bank. RSE is the Rwanda Stock Exchange. In this tutorial we'll follow the third kind, a receipt for buying shares. An ordinary invoice makes a good practice example elsewhere because it also has labels, values, and totals.

Contents

01 / Get words from a PDF **2**

02 / Find a value **3**

03 / Identify the document **4**

04 / Why JavaScript and Kotlin? **5**

05 / Run the real parser **6**

01 / Get words from a PDF

Try selecting one line

Open a PDF and try selecting and copying a sentence into a text editor. If the words paste as text, you have seen what a text layer gives the app. If you can only see a picture of the words, imagine someone photographed the receipt. The program has to recognize the letters and digits in that picture. This is called **optical character recognition**, or **OCR**.

Even a PDF with selectable text can paste in a strange order. A PDF can store *where* to draw each piece of text. It might store the amounts down a column first and the labels later. To recover the line you see, a program can group pieces that sit at about the same height, then read each group from left to right.

```
Page pieces as a reader might receive them:
  "340,728.50"        x=350, y=500
  "Net Amount Payable" x= 40, y=501

Same row, reordered from left to right:
  Net Amount Payable 340,728.50
```

In that example, X is the horizontal position and Y is the vertical position. Portfolio's Android code groups text pieces within three PDF points of the top baseline of a row, then sorts them by X. On iPhone, PDFKit usually supplies page text directly. For a recognized BK Capital statement, Portfolio can rebuild rows using the positions of individual words.

Try it with real PDF bytes. From the Portfolio repository, run the command below. The test uses the copy of PDF.js that ships with the app to read PDF bytes and send extracted text through the field parser. The next lesson starts with plain text so you can practice one step at a time.

```
node --test tests/signed-pdf-import.test.mjs
```

Portfolio tries OCR when a page yields fewer than 80 non-whitespace characters. Android uses Tesseract and iPhone uses Apple's Vision. If OCR reads a `5` as an `S`, you may see a missing field or a total that fails a check. OCR can also produce a different but valid digit, so compare the extracted values with the original page.

Find this in your code. Start with `web-logic/source/pdf-import.ts` for Android's text extraction and OCR logic. Search the iPhone file `LocalInvestmentEngine.ios.kt` for `page.string` and `recognize(page)`.

02 / Find a value

Use the label as an anchor

Suppose you already have the extracted text. Read the line as a person first. Where would you look for the amount? The words **Net Amount Payable** tell you which number matters. Looking for the first number on the page might give you a date, a share count, or an account reference.

Paste this into a terminal with Node.js installed. You can run this first example from any folder.

```
node <<'JS'
const text = [
  "No. of Shares: 500",
  "Net Amount Payable 340,728.50"
].join("\n");

const match = text.match(
  /^Net Amount Payable[ \t]+([\d,]+\.\d{2})[ \t]*$/im
);
if (!match) throw new Error("Amount line missing or unexpected");

const token = match[1];
const amount = Number(token.replaceAll(",", ""));
console.log(amount); // 340728.5
JS
```

The pattern between the slashes is a **regular expression**, a way to describe the text you expect. Here is how to read this one.

- `^Net Amount Payable` looks for the label at the beginning of a line. The `m` flag makes line boundaries count and `i` ignores capitalization.
- `[ \t]+` means one or more spaces or tabs. It keeps the search on the same line.
- The parentheses capture the amount. `[\d,]+` matches digits and commas. `\.` matches the decimal point and `\d{2}` matches two decimal digits.
- `[ \t]*$` allows spaces or tabs before the end of the line. `match[1]` gives you the part inside the parentheses.

Try changing `500` to `900`. The output stays the same because the parser anchors to the amount label. Then change the label to **Amount Due** and run it again. The error shows that your rule depends on the label. When adapting this to invoices, collect representative examples before deciding which labels and layouts to accept.

This practice pattern allows some malformed comma placements. Portfolio uses a stricter helper called `pdfNumber` to validate an entire amount token before converting it.

03 / Identify the document

Choose rules before reading the amounts

Many files say **Total**. Portfolio first looks for clues about the document itself. A BK Capital share purchase note has the broker name and a heading like **Equity Purchase Contract Note**. An equity sale note can look similar, so Portfolio checks for sale markers as well. It also checks that this purchase format uses RSE and RWF.

Think of these as two different kinds of keywords. **Document markers** tell you which recipe to use. **Field labels** tell you where to read one value once you have chosen the recipe. The shared function `parsePurchasePdfText` checks for conflicting document families and routes the text to the matching rules.

```
Document clues: BK Capital + Equity Purchase Contract Note
Field label:    Brokerage Commission
Example row:    Brokerage Commission 1.50% 5,025.00
Captured value:                          5,025.00
```

A fee row can carry two numbers. The production equity parser takes the last whole token from that row and validates it as money. A token is one chunk of text separated by whitespace. The label gives the number meaning and the layout tells you which number to capture.

Check the result before saving. The sample has 500 shares at RWF 670 each, so the gross cost is RWF 335,000. Add RWF 5,728.50 in charges and you get RWF 340,728.50. This arithmetic can catch a parsing mistake that a keyword check misses. The parser can report missing values or warnings for the review screen. It also keeps identifiers such as a trade report ID as text to preserve their digits.

Try the same method elsewhere. Take two sample invoices from one supplier. Write down the clues that identify the invoice format. Pick one field, such as the amount due. Note whether the value appears beside its label, under it, or in a table. Capture the value, check its format, then compare it with the subtotal and tax. Try a missing label and a malformed number. Treat the result as a draft until it passes your checks.

Find this in your code. `web-logic/source/equity-pdf-parser.ts` contains the dispatcher and the equity rules. `web-logic/source/document-parser-support.ts` contains shared helpers, including `pdfNumber`.

04 / Why JavaScript and Kotlin?

Let each part do the job it already knows

Kotlin runs Portfolio's mobile screens and coordinates the file picker, progress, review, and saving. The rules for recognizing investment documents are written in TypeScript, a language that adds type checks to JavaScript. A build script turns those rules into JavaScript that both mobile platforms can run.

Why keep the rules in JavaScript? Suppose BK Capital changes the label for its brokerage fee. With shared rules, you can update the label in one TypeScript parser and rebuild the bundle. Android and iPhone then interpret the new label using the same code. Kotlin could express those rules too. In this app, keeping the TypeScript rules lets the mobile app reuse code from the web side.

```
Android:
  Kotlin opens the file and sends bytes to a WebView.
  PDF.js reads the pages inside the WebView.
  The shared JavaScript rules read the resulting text.

iPhone:
  Kotlin uses Apple's PDFKit to get page text.
  Kotlin passes the text to JavaScriptCore.
  The shared JavaScript rules read the resulting text.
```

A **WebView** is an embedded browser component. Portfolio uses it to run its bundled JavaScript and the PDF.js reader on Android. **JavaScriptCore** is Apple's engine for running JavaScript. Portfolio uses it on iPhone to evaluate the field rules after extracting text.

Both call the same function. Think of it as handing a colleague a transcript of the page, with a note about whether any text was recognized from an image.

```
const result = PortfolioLogic.parse(text, usedOcr);
// text: the extracted words and line breaks
// usedOcr: true if OCR ran on any page
// result: the detected document type and extracted data
```

Kotlin receives the result as JSON, a text format for structured data, and can show the extracted values for review. PDF extraction and field parsing run on the phone. For a new purchase in the signed-in flow, confirming the import can upload the original PDF and save the reviewed purchase.

Find this in your code. Search for `PortfolioLogic.parse` in `LocalInvestmentEngine.ios.kt` and the Android engine assets. `scripts/bundle-shared-logic.mjs` builds the shared JavaScript file.

05 / Run the real parser

Follow the example into Portfolio

Open a terminal at the Portfolio repository root. The example reads a test fixture, which is sample text stored in a file. It calls the app's generated JavaScript field rules. You can use this to practice the same step without opening the mobile app.

```
node --input-type=module <<'JS'
import { readFileSync } from "node:fs";

const engine = "./app/shared/src/commonMain/composeResources/"
  + "files/engine/equity-pdf-parser.js";
const { parsePurchasePdfText } = await import(engine);

const text = readFileSync("tests/fixtures/equity-purchase.txt", "utf8");
const purchase = parsePurchasePdfText(text, false);

console.log(purchase.kind);                    // equity
console.log(purchase.fields.shares);           // 500
console.log(purchase.fields.netAmountPayable); // 340728.5
console.log(purchase.missingFields);           // []
JS
```

The `false` says this text did not come from OCR. Try `true` and add `console.log(purchase.warnings)`. You'll see a reminder to compare the recognized values with the PDF. Next, try passing `text.replace("5,025.00", "5,025.001")` to the parser. Inspect `missingFields` to see how the rules handle an amount with too many decimal places.

Try the whole flow on your phone. If you have a BK Capital RSE equity purchase contract note in RWF, open Portfolio's **Library**, tap **Add document**, and select the file. When the app offers a review, compare the quantity and net amount with the page before confirming. A file you imported before can lead to an existing import. For practice with a different kind of PDF, reuse the text exercise and choose a label from that file.

When you get stuck, locate the step. First, see whether the expected line made it out of the PDF. If the label is missing or the number is on the wrong line, inspect text extraction or OCR. If the line is correct, check the document clues and the field pattern. If the value looks plausible, verify it against the original page and any totals you can recompute.

Source walkthrough based on the Portfolio working tree at `8ff4d64`.